

Event Bus

L I T E

A LIGHTWEIGHT, TYPE-SAFE EVENT BUS FOR UNITY

DOCUMENTATION

EVENT BUS LITE

A lightweight, type-safe event bus for Unity. It lets any part of your project broadcast events and any other part listen for them **without the two ever holding a reference to each other**. No inspector wiring, no singletons of your own to maintain, no string event names.

- **Type-safe** — events are plain C# types; listeners receive the exact type they subscribed to.
- **No setup** — the bus creates itself on first use. Nothing to place in a scene.
- **Fully decoupled** — broadcasters and listeners never need to know about one another.
- **Safe by default** — a throwing listener never stops the others, and accidental re-entrant broadcasts are caught rather than crashing.
- **Leak-resistant** — a simple subscription-lifetime helper to unsubscribe everything.

Table of contents

1. Requirements
2. Installation
3. Quick start
4. Core concepts
5. The subscription lifecycle pattern
6. API reference
7. Design decisions
8. Demo scene
9. FAQ & troubleshooting
10. Support

Requirements

- Unity 6000.3.10f1 LTS or newer.
- No third-party dependencies.

Installation

1. Import the package from the Unity Asset Store into your project.
2. That's it. The API lives in the `Rosynant.EventBusLite` namespace and the `Rosynant.EventBusLite` assembly.

If your own scripts are organized under an **assembly definition**, add a reference to `Rosynant.EventBusLite` in that `asmdef` so you can see the API. If your scripts are in the default `Assembly-CSharp` (i.e. you don't use `asmdefs`), no reference step is needed.

```
using Rosynant.EventBusLite;
```

Quick start

1. Define an event

An event is any `struct` or `class` that implements the empty marker interface `IEvent`. Put whatever data you want to carry on it.

```
using Rosynant.EventBusLite;

public struct PlayerDied : IEvent
{
    public int Score;

    public PlayerDied(int score)
    {
        Score = score;
    }
}
```

2. Broadcast it

Anywhere in your code:

```
EventBusProvider.Broadcast(new PlayerDied(score: 1500));
```

3. Listen for it

```
using UnityEngine;
using Rosynant.EventBusLite;

public class GameOverScreen : MonoBehaviour, IEventSubscriber
{
    public EventSubscriptionBin EventSubscriptionBin { get; } = new();

    private void Start()
    {
        EventBusProvider
            .Subscribe<PlayerDied>(OnPlayerDied)
            .AddToSubscriptionBin(EventSubscriptionBin);
    }

    private void OnPlayerDied(PlayerDied e)
    {
        Debug.Log($"Game over! Final score: {e.Score}");
    }

    private void OnDestroy()
    {
        EventSubscriptionBin.Dispose();
    }
}
```

The broadcaster and the listener never reference each other — they only share the `PlayerDied` type. That is the whole point.

Core concepts

Type	What it is
<code>IEvent</code>	Empty marker interface. Every event type implements it.
<code>EventBusProvider</code>	The single entry point. Static <code>Subscribe<T></code> / <code>Broadcast<T></code> methods route to the right bus. You never instantiate it.
<code>EventBus<T></code>	The actual bus for one event type. Created and cached for you, per type, by the provider. You normally never touch it directly.
<code>EventSubscriptionBin</code>	Collects your subscription handles so you can dispose them all at once (typically in <code>OnDestroy</code>).

Type	What it is
<code>IEventSubscriber</code>	An optional convenience interface for classes that hold an <code>EventSubscriptionBin</code> .
<code>AddToSubscriptionBin</code>	Extension method that drops a subscription handle into a bin in one fluent call.

How routing works: `EventBusProvider` is a lazily-created, `DontDestroyOnLoad` singleton. The first time you subscribe to or broadcast a given event type, it creates one `EventBus<T>` for that type and caches it. Every subsequent call for the same type reuses that bus. Different event types are fully isolated.

The subscription lifecycle pattern

Every call to `Subscribe` returns an `IDisposable` **handle**. Disposing that handle unsubscribes the listener. If you never dispose it, the listener stays registered for the lifetime of the application — which usually means a leak when your subscribing object is destroyed.

The recommended pattern, shown in the quick start above, uses `EventSubscriptionBin`:

1. Give your class an `EventSubscriptionBin` (implementing `IEventSubscriber` documents the intent but is optional).
2. When you subscribe, pipe the returned handle into the bin with `.AddToSubscriptionBin(...)`.
3. In `OnDestroy`, call `EventSubscriptionBin.Dispose()` — this unsubscribes **every** listener you added to it, in one call.

```

public class HudController : MonoBehaviour, IEventSubscriber
{
    public EventSubscriptionBin EventSubscriptionBin { get; } = new();

    private void OnEnable()
    {
        EventBusProvider
            .Subscribe<ScoreChanged>(OnScoreChanged)
            .AddToSubscriptionBin(EventSubscriptionBin);

        EventBusProvider
            .Subscribe<HealthChanged>(OnHealthChanged)
            .AddToSubscriptionBin(EventSubscriptionBin);
    }

    private void OnScoreChanged(ScoreChanged e) { /* Do something with the event
        ↳ data */ }
    private void OnHealthChanged(HealthChanged e) { /* Do something with the event
        ↳ data */ }

    private void OnDestroy() => EventSubscriptionBin.Dispose();
}

```

You can also manage a handle yourself if you need finer control:

```

IDisposable handle = EventBusProvider.Subscribe<ScoreChanged>(OnScoreChanged);
// ... later ...
handle.Dispose(); // Unsubscribes just this one listener

```

API reference

EventBusProvider (static entry point)

```

public static IDisposable Subscribe<T>(Action<T> callback) where T : IEvent

```

Registers callback for events of type T. Returns a handle that unsubscribes the callback when disposed. **Returns null** if the provider is unavailable (for example, during application shutdown) — this is safe to pass straight into AddToSubscriptionBin, which ignores null handles.

```

public static void Broadcast<T>(T @event) where T : IEvent

```

Delivers @event to every current subscriber of type T. Does nothing if there are no subscribers, and no-ops if the provider is unavailable (for example, during shutdown).

EventSubscriptionBin

```
public void Add(IDisposable eventSubscriptionHandle)
```

Adds a handle to the bin. A null handle is ignored (with a warning), so it is safe even when `Subscribe` returned null during shutdown.

```
public void Dispose()
```

Disposes and clears every handle in the bin, unsubscribing all of its listeners. Call this in `OnDestroy`.

EventBusExtensions

```
public static void AddToSubscriptionBin(this IDisposable disposable,  
    ↪ EventSubscriptionBin bin)
```

Fluent helper that adds a subscription handle to a bin.

IEventSubscriber

```
EventSubscriptionBin EventSubscriptionBin { get; }
```

Marks a class that owns an `EventSubscriptionBin`. Optional, but documents the lifecycle contract.

EventBus<T> (advanced)

You normally never use this directly — the provider owns it. It exposes `Subscribe`, `Broadcast`, and `Reset` (which clears all listeners). Do **not** instantiate `EventBus<T>` yourself in game code; always go through `EventBusProvider` so that broadcasters and listeners share the same cached instance.

Design decisions

Subscribing the same method twice registers it once. The bus guards against duplicate subscriptions of the *same delegate*, so a listener added twice is only called once per broadcast.

Two separate lambdas are two different subscriptions. The duplicate guard compares delegate instances. Two lambda expressions (even textually identical ones) are different instances, so both get registered and both fire:

```
// These are two distinct subscriptions  
EventBusProvider.Subscribe<Foo>(e => DoThing(e));  
EventBusProvider.Subscribe<Foo>(e => DoThing(e));
```

If you need to be able to unsubscribe, or want the duplicate guard to apply, subscribe a named method (or a cached delegate) rather than an inline lambda.

Scope & threading. The bus is global to the application (it survives scene loads via `DontDestroyOnLoad`). It is intended for use on Unity's main thread, like the rest of the Unity API.

Demo scene

A runnable example lives at:

`Assets/EventBusLite/Scenes/DemoScene.unity`

It contains:

- **EventBusBroadcasterDemo** — broadcasts a `TestEvent` carrying a `Color` (optionally randomized) when its `BroadcastTestEvent()` method is invoked.
- **EventBusSubscriberDemo** — subscribes to `TestEvent` and applies the received color to a UI Image, and disposes its subscription bin in `OnDestroy`.

Open the scene, enter Play Mode, and trigger the broadcaster to see the subscriber react — with no direct reference between them. The demo scripts are a good copy-paste starting point for your own broadcasters and listeners.

FAQ & troubleshooting

My listener never fires. Check that (a) the event type broadcast is exactly the type you subscribed to, (b) you actually subscribed before the broadcast happened (not the other way around), and (c) you didn't dispose the subscription (or its bin) prematurely.

My listener fires twice. You almost certainly subscribed with two separate lambda expressions. Subscribe a named method so the duplicate guard can recognize it, or make sure you only subscribe once. See Design decisions.

I get warnings about a re-entrant broadcast. Something is calling `Broadcast` from inside a listener for the same event type. Move that follow-up work out of the listener.

Do I have to unsubscribe? Yes, if the subscribing object can be destroyed while the app keeps running (which is almost always the case for `MonoBehaviours`). Use the `EventSubscriptionBin` pattern and dispose it in `OnDestroy`.

Can events be classes instead of structs? Yes. Any `struct` or `class` that implements `IEvent` works. Structs avoid an allocation per broadcast, which is why the demo uses one.

Do I need to add anything to my scene? No. The provider creates itself the first time you subscribe or broadcast.

Support

For questions, bug reports, or feature requests, contact me: wojciech_grodzicki@outlook.com

Please include your Unity version and a description of what you expected versus what happened.